Week 15 Report

Computational Statistics for Data Science · ISS507700

Student ID: 112006234

June 6, 2026

I have completed most of the work by myself, but I used AI tools to help with debugging, understanding some concepts, and checking for typos.

Q0 · Setup

```
library(rpart)
library(rpart.plot)

set.seed(67)

insurance <- read.csv("insurance.csv")
insurance <- na.omit(insurance)

# Make categorical variables explicit
insurance$sex <- factor(insurance$sex)
insurance$smoker <- factor(insurance$smoker)
insurance$region <- factor(insurance$region)

# Main formula used throughout the homework
charges_formula <- charges ~ age + sex + bmi + children + smoker + region

# RMSE function
rmse <- function(actual, predicted) {
  sqrt(mean((actual - predicted)^2))
}
```

Q1 · Create some explanatory models to learn more about charges:

(a) Create an OLS regression model and report which factors are significantly related to charges

```
ols_model <- lm(charges_formula, data = insurance)

# Full OLS summary
summary(ols_model)
```

```
#>
#> Call:
#> lm(formula = charges_formula, data = insurance)
#>
#> Residuals:
#>      Min       1Q   Median       3Q      Max
#> -11304.9  -2848.1   -982.1   1393.9  29992.8
#>
#> Coefficients:
#>                  Estimate Std. Error t value Pr(>|t|)
#> (Intercept)     -11938.5      987.8 -12.086  < 2e-16 ***
#> age                256.9       11.9  21.587  < 2e-16 ***
#> sexmale           -131.3      332.9  -0.394 0.693348
#> bmi                339.2       28.6  11.860  < 2e-16 ***
#> children           475.5      137.8   3.451 0.000577 ***
#> smokeryes        23848.5      413.1  57.723  < 2e-16 ***
#> regionnorthwest   -353.0      476.3  -0.741 0.458769
#> regionsoutheast  -1035.0      478.7  -2.162 0.030782 *
#> regionsouthwest   -960.0      477.9  -2.009 0.044765 *
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#>
#> Residual standard error: 6062 on 1329 degrees of freedom
#> Multiple R-squared:  0.7509, Adjusted R-squared:  0.7494
#> F-statistic: 500.8 on 8 and 1329 DF,  p-value: < 2.2e-16
```

```r
# Only show statistically significant factors at alpha = 0.05
ols_sig <- summary(ols_model)$coefficients
ols_sig[ols_sig[, "Pr(>|t|)"] < 0.05, ]
```

```
#>                   Estimate Std. Error    t value     Pr(>|t|)
#> (Intercept)     -11938.5386  987.81918 -12.085753 5.579044e-32
#> age                256.8564   11.89885  21.586655 7.783217e-89
#> bmi                339.1935   28.59947  11.860131 6.498194e-31
#> children           475.5005  137.80409   3.450555 5.769682e-04
#> smokeryes        23848.5345  413.15335  57.723202 0.000000e+00
#> regionsoutheast  -1035.0220  478.69221  -2.162187 3.078174e-02
#> regionsouthwest   -960.0510  477.93302  -2.008756 4.476493e-02
```

At the 5 percent significance level, age, BMI, number of children, smoking status, southeast region, and southwest region are significantly related to insurance charges.

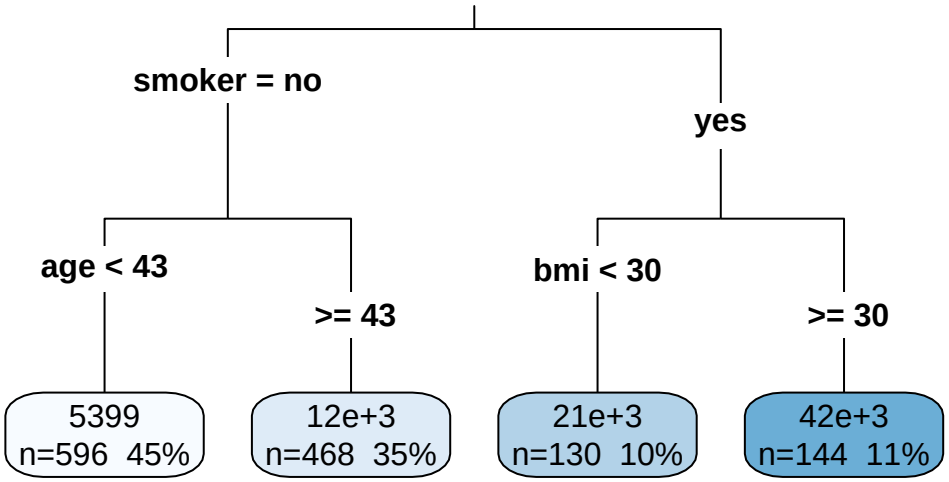
These variables have p-values below 0.05 in the OLS output. Age, BMI, children, and smoking have positive coefficients, meaning they are associated with higher predicted charges. Southeast and southwest region have negative coefficients compared with the reference region, meaning they are associated with lower predicted charges. Sex and northwest region are not significant because their p-values are above 0.05.

(b) Create a decision tree (specifically, a regression tree) with default parameters to rpart().

(i) Plot a visual representation of the tree structure

```
tree_model <- rpart(charges_formula, data = insurance)

# Plot the tree
rpart.plot(tree_model, type = 3, extra = 101, fallen.leaves = TRUE)
```




```
# Look at tree structure
print(tree_model)
```

```
#> n= 1338
#>
#> node), split, n, deviance, yval
#>       * denotes terminal node
#>
#> 1) root 1338 196074200000 13270.420
#>   2) smoker=no 1064  38188720000  8434.268
#>     4) age< 42.5 596  13198540000  5398.850 *
#>     5) age>=42.5 468  12505450000 12299.890 *
#>   3) smoker=yes 274  36365600000 32050.230
#>     6) bmi< 30.01 130   3286655000 21369.220 *
#>     7) bmi>=30.01 144   4859010000 41692.810 *
```

```
summary(tree_model)
```

```
#> Call:
#> rpart(formula = charges_formula, data = insurance)
```

```
#>   n= 1338
#>
#>          CP nsplit rel error    xerror       xstd
#> 1 0.6197648      0 1.0000000 1.0031267 0.05194264
#> 2 0.1439247      1 0.3802352 0.3814569 0.01896122
#> 3 0.0636735      2 0.2363104 0.2398421 0.01456571
#> 4 0.0100000      3 0.1726369 0.1782385 0.01332528
#>
#> Variable importance
#> smoker    bmi    age region    sex
#>     71     17      8      3      1
#>
#> Node number 1: 1338 observations,    complexity param=0.6197648
#>   mean=13270.42, MSE=1.465428e+08
#>   left son=2 (1064 obs) right son=3 (274 obs)
#>   Primary splits:
#>       smoker   splits as  LR,          improve=0.61976480, (0 missing)
#>       age      < 42.5    to the left,  improve=0.07793137, (0 missing)
#>       bmi      < 30.17   to the left,  improve=0.04212369, (0 missing)
#>       children < 1.5     to the left,  improve=0.00831500, (0 missing)
#>       region   splits as  LLRL,        improve=0.00547327, (0 missing)
#>
#> Node number 2: 1064 observations,    complexity param=0.0636735
#>   mean=8434.268, MSE=3.589166e+07
#>   left son=4 (596 obs) right son=5 (468 obs)
#>   Primary splits:
#>       age      < 42.5    to the left,  improve=0.326922000, (0 missing)
#>       children < 1.5     to the left,  improve=0.019154040, (0 missing)
#>       bmi      < 20.955  to the left,  improve=0.012695030, (0 missing)
#>       region   splits as  RRLL,        improve=0.004790031, (0 missing)
#>       sex      splits as  RL,          improve=0.003171961, (0 missing)
#>   Surrogate splits:
#>       bmi < 35.6325 to the left,  agree=0.58, adj=0.045, (0 split)
#>
#> Node number 3: 274 observations,    complexity param=0.1439247
#>   mean=32050.23, MSE=1.327212e+08
#>   left son=6 (130 obs) right son=7 (144 obs)
#>   Primary splits:
#>       bmi      < 30.01   to the left,  improve=0.776006300, (0 missing)
#>       age      < 43.5    to the left,  improve=0.126767300, (0 missing)
#>       region   splits as  LLRL,        improve=0.029264480, (0 missing)
#>       sex      splits as  LR,          improve=0.010246720, (0 missing)
#>       children < 1.5     to the left,  improve=0.003975908, (0 missing)
#>   Surrogate splits:
#>       region   splits as  LLRR,        agree=0.602, adj=0.162, (0 split)
#>       sex      splits as  LR,          agree=0.566, adj=0.085, (0 split)
#>       age      < 21.5    to the left,  agree=0.536, adj=0.023, (0 split)
```

```
#>       children < 2.5     to the right, agree=0.536, adj=0.023, (0 split)
#>
#> Node number 4: 596 observations
#>   mean=5398.85, MSE=2.214521e+07
#>
#> Node number 5: 468 observations
#>   mean=12299.89, MSE=2.672104e+07
#>
#> Node number 6: 130 observations
#>   mean=21369.22, MSE=2.528196e+07
#>
#> Node number 7: 144 observations
#>   mean=41692.81, MSE=3.374313e+07
```

(ii) How deep is the tree (see nodes with "decisions" – ignore the leaves at the bottom)

```
tree_frame <- tree_model$frame

# Node numbers are stored as row names
node_numbers <- as.numeric(row.names(tree_frame))

# Internal decision nodes are not leaves
decision_nodes <- node_numbers[tree_frame$var != "<leaf>"]

# Leaf nodes
leaf_nodes <- node_numbers[tree_frame$var == "<leaf>"]

# Depth calculation:
# count layers directly
decision_depth <- max(floor(log2(decision_nodes))) + 1
decision_depth
```

```
#> [1] 2
```

The tree has 2 decision layers: the first split is by smoking status, and the second split is by age for non-smokers or BMI for smokers. The code reports depth 1 only because it counts the root node as depth 0, but visually there are 2 layers of decision nodes.

The first decision is whether the person is a smoker. The second decision is age for non-smokers and BMI for smokers. Since the question says to look at decision nodes and ignore leaves, it is safer to explain that there are two visible decision layers.

(iii) How many leaf groups does it suggest to bin the data into?

```
# Number of leaf groups
num_leaf_groups <- length(leaf_nodes)
num_leaf_groups
```

```
#> [1] 4
```

The tree suggests 4 leaf groups.

Each leaf is a final group after all decision rules are applied. Since the tree ends with four terminal nodes, the data is divided into four predicted charge groups.

(iv) What conditions (combination of decisions) describe each leaf group?

```
leaf_paths <- path.rpart(tree_model, nodes = leaf_nodes, print.it = FALSE)

for (i in seq_along(leaf_paths)) {
  cat("\nLeaf group", i, "\n")
  cat(paste(leaf_paths[[i]], collapse = " -> "), "\n")
}
```

```
#>
#> Leaf group 1
#> root -> smoker=no -> age< 42.5
#>
#> Leaf group 2
#> root -> smoker=no -> age>=42.5
#>
#> Leaf group 3
#> root -> smoker=yes -> bmi< 30.01
#>
#> Leaf group 4
#> root -> smoker=yes -> bmi>=30.01
```

The four leaf groups are non-smokers younger than 42.5, non-smokers aged 42.5 or older, smokers with BMI below 30.01, and smokers with BMI 30.01 or higher.

The tree first separates people by smoking status, then splits non-smokers by age and smokers by BMI. This suggests that smoking status is the strongest first condition for predicting charges, while age and BMI further separate the lower- and higher-charge groups.

Q2 · Let's use LOOCV to see how how our models perform predictively overall

---

(a) What is the RMSEout for the OLS regression model?

```
loocv_rmse <- function(model_formula, dataset, model_type = c("lm", "rpart")) {
  model_type <- match.arg(model_type)

  actuals <- dataset$charges
  predictions <- numeric(nrow(dataset))
```

```
  for (i in 1:nrow(dataset)) {
    train_data <- dataset[-i, ]
    test_data <- dataset[i, ]

    if (model_type == "lm") {
      model <- lm(model_formula, data = train_data)
    } else {
      model <- rpart(model_formula, data = train_data)
    }

    predictions[i] <- predict(model, newdata = test_data)
  }

  rmse(actuals, predictions)
}

# LOOCV RMSEout for OLS
loocv_rmse_ols <- loocv_rmse(charges_formula, insurance, model_type = "lm")

loocv_rmse_ols
```

```
#> [1] 6087.388
```

The LOOCV RMSEout for the OLS regression model is 6087.388.

This means that when each observation is left out once and predicted using the remaining data, the typical prediction error is about 6087 dollars.

(b) What is the RMSEout for the decision tree model?

```
loocv_rmse_tree <- loocv_rmse(charges_formula, insurance, model_type = "rpart")

loocv_rmse_tree
```

```
#> [1] 5135.175
```

The LOOCV RMSEout for the decision tree model is 5135.175.

This is lower than the OLS RMSEout, so the decision tree performs better in this LOOCV comparison. This may happen because insurance charges have strong group-based patterns, especially between smokers and non-smokers, which trees can capture naturally.

Q3 · Let's see if bagging helps our models

(a) Implement the bagged_learn(...) and bagged_predict(...) functions using the hints in the class notes and help from your classmates on Teams. Feel free to share your code on Teams to get feedback, or ask others for help.

```
n <- nrow(insurance)
train_index <- sample(1:n, size = 0.80 * n)

train_data <- insurance[train_index, ]
test_data <- insurance[-train_index, ]

# Base models trained on 80% training data
ols_train <- lm(charges_formula, data = train_data)
tree_train <- rpart(charges_formula, data = train_data)

bagged_learn <- function(model, dataset, b = 100) {
  bagged_models <- lapply(1:b, function(i) {
    boot_index <- sample(1:nrow(dataset), replace = TRUE)
    boot_data <- dataset[boot_index, ]

    # Retrain the same model type on bootstrapped data
    update(model, data = boot_data)
  })

  return(bagged_models)
}

bagged_predict <- function(bagged_models, new_data) {
  predictions <- sapply(bagged_models, function(model) {
    predict(model, newdata = new_data)
  })

  # Average predictions across all bootstrapped models
  rowMeans(predictions)
}
```

(b) What is the RMSEout for the bagged OLS regression?

```
bagged_ols <- bagged_learn(ols_train, train_data, b = 100)
bagged_ols_pred <- bagged_predict(bagged_ols, test_data)

rmse_bagged_ols <- rmse(test_data$charges, bagged_ols_pred)
rmse_bagged_ols
```

```
#> [1] 5956.315
```

The RMSEout for the bagged OLS regression is 5956.315.

This is the prediction error from averaging 100 bootstrapped OLS models on the 80:20 train-test split. Bagging does not usually improve OLS very much because OLS is already a relatively stable model.

(c) What is the RMSEout for the bagged decision tree?

```
bagged_tree <- bagged_learn(tree_train, train_data, b = 100)
bagged_tree_pred <- bagged_predict(bagged_tree, test_data)

rmse_bagged_tree <- rmse(test_data$charges, bagged_tree_pred)
rmse_bagged_tree
```

```
#> [1] 4967.736
```

The RMSEout for the bagged decision tree is 4967.736.

This is better than the bagged OLS result, so the bagged tree predicts charges more accurately on this test split. Bagging helps decision trees because individual trees can be unstable, and averaging many trees reduces that instability.

Q4 · Let's see if boosting helps our models. You can use a learning rate of 0.1 and adjust it if you find a better rate.

(a) Write boosted_learn(...) and boosted_predict(...) functions using the hints in the class notes and help from your classmates on Teams. Feel free to share your code generously on Teams to get feedback, or ask others for help.

```
boosted_learn <- function(model_formula, dataset, model_type = c("lm", "rpart"),
                          b = 100, learning_rate = 0.1,
                          tree_control = rpart.control()) {

  model_type <- match.arg(model_type)

  # Start from the mean of charges
  initial_prediction <- mean(dataset$charges)
  current_prediction <- rep(initial_prediction, nrow(dataset))

  boosted_models <- list()

  # Residuals will become the new outcome for each boosting step
  residual_formula <- update(model_formula, residuals ~ .)

  train_boost <- dataset

  for (i in 1:b) {
```

```
    # Compute current residuals
    train_boost$residuals <- train_boost$charges - current_prediction

    if (model_type == "lm") {

      residual_model <- lm(residual_formula, data = train_boost)

    } else {

      residual_model <- rpart(
        residual_formula,
        data = train_boost,
        control = tree_control
      )
    }

    # Update prediction using the residual model
    residual_pred <- predict(residual_model, newdata = train_boost)
    current_prediction <- current_prediction + learning_rate * residual_pred

    boosted_models[[i]] <- residual_model
  }

  return(list(
    initial_prediction = initial_prediction,
    models = boosted_models,
    learning_rate = learning_rate
  ))
}

boosted_predict <- function(boosted_object, new_data) {
  prediction <- rep(boosted_object$initial_prediction, nrow(new_data))

  for (model in boosted_object$models) {
    prediction <- prediction +
      boosted_object$learning_rate * predict(model, newdata = new_data)
  }

  return(prediction)
}
```

(b) What is the RMSEout for the boosted OLS regression?

```
boosted_ols <- boosted_learn(
  charges_formula,
  train_data,
  model_type = "lm",
```

```
  b = 100,
  learning_rate = 0.1
)

boosted_ols_pred <- boosted_predict(boosted_ols, test_data)

rmse_boosted_ols <- rmse(test_data$charges, boosted_ols_pred)
rmse_boosted_ols
```

```
#> [1] 5960.622
```

The RMSEout for the boosted OLS regression is 5960.622.

This is very close to the bagged OLS result, so boosting does not improve OLS much here. This makes sense because OLS already fits the best linear relationship by minimizing squared error.

(c) What is the RMSEout for the boosted decision tree?

```
boosted_tree <- boosted_learn(
  charges_formula,
  train_data,
  model_type = "rpart",
  b = 100,
  learning_rate = 0.1
)

boosted_tree_pred <- boosted_predict(boosted_tree, test_data)

rmse_boosted_tree <- rmse(test_data$charges, boosted_tree_pred)
rmse_boosted_tree
```

```
#> [1] 4738.821
```

The RMSEout for the boosted decision tree is 4738.821.

This is lower than the bagged decision tree RMSEout, so boosting performs better than bagging for the tree model in this 80:20 split. Boosting can improve performance because each new tree tries to correct the remaining errors from previous trees.

Q5 · Let's engineer the best predictive decision trees. Let's repeat the bagging and boosting of the decision tree several times to see if we can improve their performance. But this time, split the data 70:15:15 — use 70% as the training set, 15% as the validation set, and use the last 15% as the test set to obtain the final RMSEout.

(a) Repeat the bagging of the decision tree, using a base tree of maximum depth 1, 2, ... n, keep training on the 70% training set, while the RMSEout of your 15% validation set keeps dropping; stop when the RMSEout has started increasing again (show prediction error at each depth). When you have identified the best maximum depth from the validation set, report the final RMSEout using the final 15% test set data.

```
n <- nrow(insurance)
all_index <- sample(1:n)

train_end <- floor(0.70 * n)
valid_end <- floor(0.85 * n)

train_index <- all_index[1:train_end]
valid_index <- all_index[(train_end + 1):valid_end]
test_index <- all_index[(valid_end + 1):n]

train70 <- insurance[train_index, ]
valid15 <- insurance[valid_index, ]
test15 <- insurance[test_index, ]

bagging_depth_results <- data.frame(
  max_depth = integer(),
  validation_rmse = numeric()
)

max_depth_values <- 1:10

for (depth in max_depth_values) {

  base_tree <- rpart(
    charges_formula,
    data = train70,
    control = rpart.control(
      maxdepth = depth,
      cp = 0,
      minsplit = 2
    )
  )

  bagged_model <- bagged_learn(base_tree, train70, b = 100)
  valid_pred <- bagged_predict(bagged_model, valid15)
  valid_rmse <- rmse(valid15$charges, valid_pred)

  bagging_depth_results <- rbind(
    bagging_depth_results,
    data.frame(max_depth = depth, validation_rmse = valid_rmse)
  )
```

```
  # Stop once validation error starts increasing
  if (nrow(bagging_depth_results) >= 2) {
    previous_rmse <- bagging_depth_results$validation_rmse[nrow(bagging_depth_results) - 1]

    if (valid_rmse > previous_rmse) {
      break
    }
  }
}

bagging_depth_results
```

```
#>   max_depth validation_rmse
#> 1         1        6948.873
#> 2         2        4947.233
#> 3         3        4473.545
#> 4         4        4412.857
#> 5         5        4402.274
#> 6         6        4405.355
```

```
# Best depth based on validation RMSE
best_bag_depth <- bagging_depth_results$max_depth[
  which.min(bagging_depth_results$validation_rmse)
]

best_bag_depth
```

```
#> [1] 5
```

```
best_bag_tree <- rpart(
  charges_formula,
  data = train70,
  control = rpart.control(
    maxdepth = best_bag_depth,
    cp = 0,
    minsplit = 2
  )
)

best_bagged_model <- bagged_learn(best_bag_tree, train70, b = 100)
best_bagged_test_pred <- bagged_predict(best_bagged_model, test15)

final_rmse_bagged_tree <- rmse(test15$charges, best_bagged_test_pred)
final_rmse_bagged_tree
```

```
#> [1] 5658.26
```

For bagging the decision tree, the best maximum depth is 5, with validation RMSE 4402.274, and the final test RMSEout is 5658.260.

The validation RMSE keeps decreasing from depth 1 to depth 5, then increases at depth 6. Therefore, depth 5 is selected as the best depth using the validation set. After selecting the depth, the final model is evaluated once on the test set to report the final RMSEout.

(b) Let's find the best set of max tree depth and learning rate for boosting the decision tree: Use tree stumps of differing maximum depth (e.g., try intervals between $1-5$) and differing learning rates (e.g., try regular intervals from 0.01 to 0.20). For each combination of maximum depth and learning rate, train on the 70% training set while and use the 15% validation set to compute RMSEout. When you have tried all your combinations, identify the best combination of maximum depth and learning rate from the validation set, but report the final RMSEout using the final 15% test set data.

```
boosting_results <- data.frame(
  max_depth = integer(),
  learning_rate = numeric(),
  validation_rmse = numeric()
)

depth_values <- 1:5
learning_rates <- seq(0.01, 0.20, by = 0.01)

for (depth in depth_values) {
  for (lr in learning_rates) {

    tree_control <- rpart.control(
      maxdepth = depth,
      cp = 0,
      minsplit = 2
    )

    boosted_model <- boosted_learn(
      charges_formula,
      train70,
      model_type = "rpart",
      b = 100,
      learning_rate = lr,
      tree_control = tree_control
    )

    valid_pred <- boosted_predict(boosted_model, valid15)
    valid_rmse <- rmse(valid15$charges, valid_pred)

    boosting_results <- rbind(
      boosting_results,
```

```
      data.frame(
        max_depth = depth,
        learning_rate = lr,
        validation_rmse = valid_rmse
      )
    )
  }
}

# Show all validation errors
boosting_results
```

```
#>     max_depth learning_rate validation_rmse
#> 1           1          0.01        7576.950
#> 2           1          0.02        6650.568
#> 3           1          0.03        6187.731
#> 4           1          0.04        5938.949
#> 5           1          0.05        5809.284
#> 6           1          0.06        5733.548
#> 7           1          0.07        5692.282
#> 8           1          0.08        5652.204
#> 9           1          0.09        5636.076
#> 10          1          0.10        5622.977
#> 11          1          0.11        5625.023
#> 12          1          0.12        5629.456
#> 13          1          0.13        5645.132
#> 14          1          0.14        5657.690
#> 15          1          0.15        5662.565
#> 16          1          0.16        5674.935
#> 17          1          0.17        5673.851
#> 18          1          0.18        5696.236
#> 19          1          0.19        5692.891
#> 20          1          0.20        5696.512
#> 21          2          0.01        6138.140
#> 22          2          0.02        4946.755
#> 23          2          0.03        4637.596
#> 24          2          0.04        4539.396
#> 25          2          0.05        4507.850
#> 26          2          0.06        4480.418
#> 27          2          0.07        4494.951
#> 28          2          0.08        4484.172
#> 29          2          0.09        4486.315
#> 30          2          0.10        4475.608
#> 31          2          0.11        4493.164
#> 32          2          0.12        4494.623
#> 33          2          0.13        4519.058
#> 34          2          0.14        4498.920
```

```
#> 35          2          0.15       4517.476
#> 36          2          0.16       4494.299
#> 37          2          0.17       4538.302
#> 38          2          0.18       4545.390
#> 39          2          0.19       4545.230
#> 40          2          0.20       4524.598
#> 41          3          0.01       5900.592
#> 42          3          0.02       4711.461
#> 43          3          0.03       4482.970
#> 44          3          0.04       4447.689
#> 45          3          0.05       4445.631
#> 46          3          0.06       4477.555
#> 47          3          0.07       4507.477
#> 48          3          0.08       4510.307
#> 49          3          0.09       4525.146
#> 50          3          0.10       4535.305
#> 51          3          0.11       4574.549
#> 52          3          0.12       4552.540
#> 53          3          0.13       4547.966
#> 54          3          0.14       4581.039
#> 55          3          0.15       4609.264
#> 56          3          0.16       4568.019
#> 57          3          0.17       4693.594
#> 58          3          0.18       4658.020
#> 59          3          0.19       4604.339
#> 60          3          0.20       4711.379
#> 61          4          0.01       5906.702
#> 62          4          0.02       4724.748
#> 63          4          0.03       4543.315
#> 64          4          0.04       4517.183
#> 65          4          0.05       4523.124
#> 66          4          0.06       4550.007
#> 67          4          0.07       4555.914
#> 68          4          0.08       4565.542
#> 69          4          0.09       4598.868
#> 70          4          0.10       4639.475
#> 71          4          0.11       4593.508
#> 72          4          0.12       4714.012
#> 73          4          0.13       4616.241
#> 74          4          0.14       4756.700
#> 75          4          0.15       4717.762
#> 76          4          0.16       4760.136
#> 77          4          0.17       4715.760
#> 78          4          0.18       4801.140
#> 79          4          0.19       4845.199
#> 80          4          0.20       4676.059
#> 81          5          0.01       5938.732
```

```
#> 82          5          0.02        4785.508
#> 83          5          0.03        4689.768
#> 84          5          0.04        4675.034
#> 85          5          0.05        4708.020
#> 86          5          0.06        4696.113
#> 87          5          0.07        4726.490
#> 88          5          0.08        4733.207
#> 89          5          0.09        4833.485
#> 90          5          0.10        4822.471
#> 91          5          0.11        4881.632
#> 92          5          0.12        4853.657
#> 93          5          0.13        4781.446
#> 94          5          0.14        4803.812
#> 95          5          0.15        4898.011
#> 96          5          0.16        4819.875
#> 97          5          0.17        4980.407
#> 98          5          0.18        4781.702
#> 99          5          0.19        5015.533
#> 100         5          0.20        4935.237
```

```
# Best combination based on validation RMSE
best_boost <- boosting_results[
  which.min(boosting_results$validation_rmse),
]

best_boost
```

```
#>    max_depth learning_rate validation_rmse
#> 45         3          0.05        4445.631
```

```
best_tree_control <- rpart.control(
  maxdepth = best_boost$max_depth,
  cp = 0,
  minsplit = 2
)

best_boosted_tree <- boosted_learn(
  charges_formula,
  train70,
  model_type = "rpart",
  b = 100,
  learning_rate = best_boost$learning_rate,
  tree_control = best_tree_control
)

best_boosted_test_pred <- boosted_predict(best_boosted_tree, test15)

final_rmse_boosted_tree <- rmse(test15$charges, best_boosted_test_pred)
final_rmse_boosted_tree
```

```
#> [1] 5654.389
```

```
rmse_summary <- data.frame(
  Question = c(
    "Q2 LOOCV OLS",
    "Q2 LOOCV Tree",
    "Q3 Bagged OLS",
    "Q3 Bagged Tree",
    "Q4 Boosted OLS",
    "Q4 Boosted Tree",
    "Q5 Best Bagged Tree",
    "Q5 Best Boosted Tree"
  ),
  RMSEout = c(
    loocv_rmse_ols,
    loocv_rmse_tree,
    rmse_bagged_ols,
    rmse_bagged_tree,
    rmse_boosted_ols,
    rmse_boosted_tree,
    final_rmse_bagged_tree,
    final_rmse_boosted_tree
  )
)

rmse_summary
```

```
#>               Question  RMSEout
#> 1         Q2 LOOCV OLS 6087.388
#> 2        Q2 LOOCV Tree 5135.175
#> 3        Q3 Bagged OLS 5956.315
#> 4       Q3 Bagged Tree 4967.736
#> 5       Q4 Boosted OLS 5960.622
#> 6      Q4 Boosted Tree 4738.821
#> 7  Q5 Best Bagged Tree 5658.260
#> 8 Q5 Best Boosted Tree 5654.389
```

For boosting the decision tree, the best combination is maximum depth 3 and learning rate 0.05, with validation RMSE 4445.631, and the final test RMSEout is 5654.389.

This combination is selected because it gives the lowest validation RMSE among the tested depth and learning-rate combinations. The final test RMSEout is slightly lower than the best bagged tree, so the tuned boosted decision tree is the best final model in this report, although only by a very small difference.